

Bài 2: NGÔN NGỮ LẬP TRÌNH HƯỚNG ĐỐI TƯỢNG JAVA

Mục tiêu

Về kiến thức:

- + Hiểu được các thành phần cơ bản của ngôn ngữ Java
- + Hiểu được các kiểu dữ liệu nguyên thủy, cấu trúc dữ liệu mảng và các câu lệnh điều khiển trong Java

Về kỹ năng:

- + Vận dụng các cấu trúc điều khiển thao tác với các kiểu dữ liệu nguyên thủy

2.1 Sơ lược lịch sử hình thành và phát triển của Java

Java là ngôn ngữ lập trình hướng đối tượng (tựa C++) do Sun Microsystem đưa ra vào giữa thập niên 90. Chương trình viết bằng ngôn ngữ lập trình java có thể chạy trên bất kỳ hệ thống nào có cài máy ảo java

Năm 1990: James Gosling và các công sự của Công ty Sun Microsystem tham gia dự án Green Team - xây dựng công nghệ mới cho ngành điện tử tiêu dung (cho các thiết bị điện dân dụng). Để giải quyết vấn đề này nhóm nghiên cứu phát triển đã xây dựng một ngôn ngữ lập trình mới đặt tên là Oak tương tự như C++ nhưng loại bỏ một số tính năng nguy hiểm của C++ và có khả năng chạy trên nhiều nền phần cứng khác nhau.

Năm 1993 world wide web bắt đầu phát triển. Năm 1994, Sun đưa ra trình duyệt web viết bằng ngôn ngữ oak là webrunner, sau đổi tên thành hostJava.

Sau đó, Sun đổi tên oak thành Java và được giới thiệu năm 1995 tại Sunworld 1995.

Java là tên gọi của một hòn đảo ở Indonexia, đây là nơi nhóm nghiên cứu phát triển đã chọn để đặt tên cho ngôn ngữ lập trình Java trong một chuyến đi tham quan và làm việc trên hòn đảo này. Hòn đảo Java này là nơi rất nổi tiếng với nhiều khu vườn trồng cafe, đó chính là lý do chúng ta thường thấy biểu tượng ly café trong nhiều sản phẩm phần mềm, công cụ lập trình Java của Sun cũng như một số hãng phần mềm khác đưa ra.

2.2 Ngữ cảnh các ứng dụng của ngôn ngữ java.

Ngôn ngữ Java có những đặc trưng cơ bản sau:

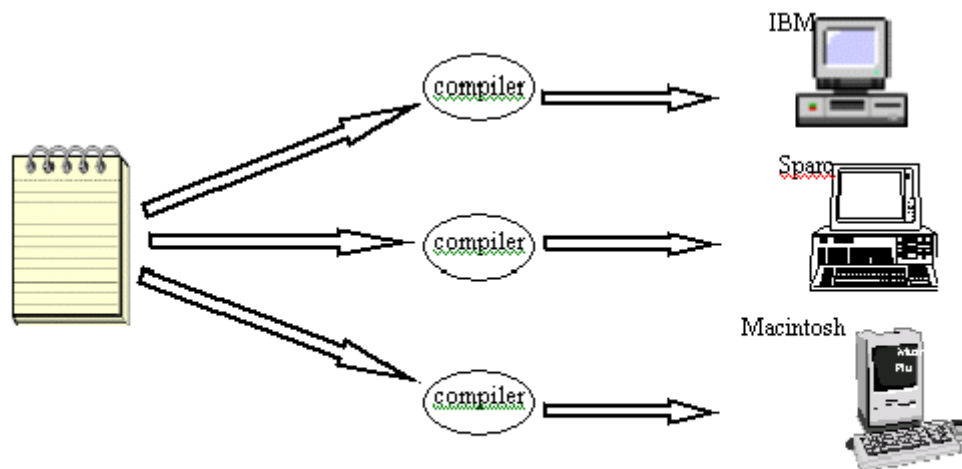
Đơn giản: phát triển trên một ngôn ngữ dễ học và quen thuộc với đa số người lập trình là C và Java loại bỏ các phức tạp của C và C++ như: con trỏ, đa kế thừa, định nghĩa chồng toán tử, không sử dụng lệnh “goto” cũng như file header (.h), loại bỏ cấu trúc “struct” và “union”.

Hướng đối tượng Java là ngôn ngữ lập trình hoàn toàn hướng đối tượng:

- Mọi thực thể trong hệ thống đều được coi là một đối tượng, tức là một thể hiện cụ thể của một lớp xác định.
- Tất cả các chương trình đều phải nằm trong một class nhất định.
- Không thể dùng Java để viết một chức năng mà không thuộc vào bất kì một lớp nào. Tức là Java không cho phép định nghĩa dữ liệu và hàm tự do trong chương trình.

Độc lập phần cứng và hệ điều hành

Đối với các ngôn ngữ lập trình truyền thống như C/C++, phương pháp biên dịch được thực hiện như sau:



Hình 2.1 Chương trình java chạy trên mọi hệ điều hành

Với một nền phần cứng khác nhau, có một trình biên dịch khác nhau để biên dịch mã nguồn chương trình cho phù hợp với nền phần cứng ấy. Do vậy, khi chạy trên một nền phần cứng khác, bắt buộc phải biên dịch lại mã nguồn.

Đối các chương trình viết bằng Java, chỉ cần cài máy ảo java là có thể chạy được.

Máy ảo Java (Java Virtual Machine).

- Là một phần mềm dựa trên cơ sở máy tính ảo
- Là tập hợp các lệnh logic để xác định hoạt động của máy tính
- Được xem như là một hệ điều hành thu nhỏ
- Nó thiết lập lớp trừu tượng cho:
 - Phần cứng bên dưới
 - Hệ điều hành
 - Mã đã biên dịch

Mạnh mẽ Java là ngôn ngữ yêu cầu chặt chẽ về kiểu dữ liệu:

- Kiểu dữ liệu phải được khai báo tường minh.
- Java không sử dụng con trỏ và các phép toán con trỏ.
- Java kiểm tra việc truy nhập đến mảng, chuỗi khi thực thi để đảm bảo rằng các truy nhập đó không ra ngoài giới hạn kích thước mảng.
- Quá trình cấp phát, giải phóng bộ nhớ cho biến được thực hiện tự động, nhờ dịch vụ thu nhặt những đối tượng không còn sử dụng nữa (garbage collection).
- Cơ chế bắt lỗi của Java giúp đơn giản hóa quá trình xử lý lỗi và hồi phục sau lỗi.

Bảo mật Java cung cấp một môi trường quản lý thực thi chương trình với nhiều mức để kiểm soát tính an toàn:

- Ở mức thứ nhất, dữ liệu và các phương thức được đóng gói bên trong lớp. Chúng chỉ được truy xuất thông qua các giao diện mà lớp cung cấp.
- Ở mức thứ hai, trình biên dịch kiểm soát để đảm bảo mã là an toàn, và tuân theo các nguyên tắc của Java.
- Mức thứ ba được đảm bảo bởi trình thông dịch. Chúng kiểm tra xem bytecode có đảm bảo các qui tắc an toàn trước khi thực thi.
- Mức thứ tư kiểm soát việc nạp các lớp vào bộ nhớ để giám sát việc vi phạm giới hạn truy xuất trước khi nạp vào hệ thống.

Phân tán Java được thiết kế để hỗ trợ các ứng dụng chạy trên mạng bằng các lớp Mạng (java.net). Hơn nữa, Java hỗ trợ nhiều nền chạy khác nhau.

Đa luồng Java cung cấp giải pháp đa luồng (Multithreading) để thực thi các công việc cùng đồng thời và đồng bộ giữa các luồng.

Linh động Java được thiết kế như một ngôn ngữ động để đáp ứng cho những môi trường mở. Các chương trình Java chứa rất nhiều thông tin thực thi nhằm kiểm soát và truy nhập đối tượng lúc chạy.

2.3. Cấu trúc chung của chương trình java

Cấu trúc chung:

```
package packageName; // Khai báo tên gói, nếu có
import java.awt.*; // Khai báo tên thư viện sẵn có, nếu cần dùng
class className // Khai báo tên lớp
{
    /* Đây là dòng ghi chú */
    int var; // Khai báo biến
```

```

public void methodName() // Khai báo tên phương thức
{
    /* Phần thân của phương thức */
    statement (s); // Lệnh thực hiện}
}

interface ...// Khai báo giao diện nếu có

```

Ví dụ minh họa

```

public class HelloWorldApp{
    public static void main(String[] args){
        System.out.println("HelloWorld");
    }
}

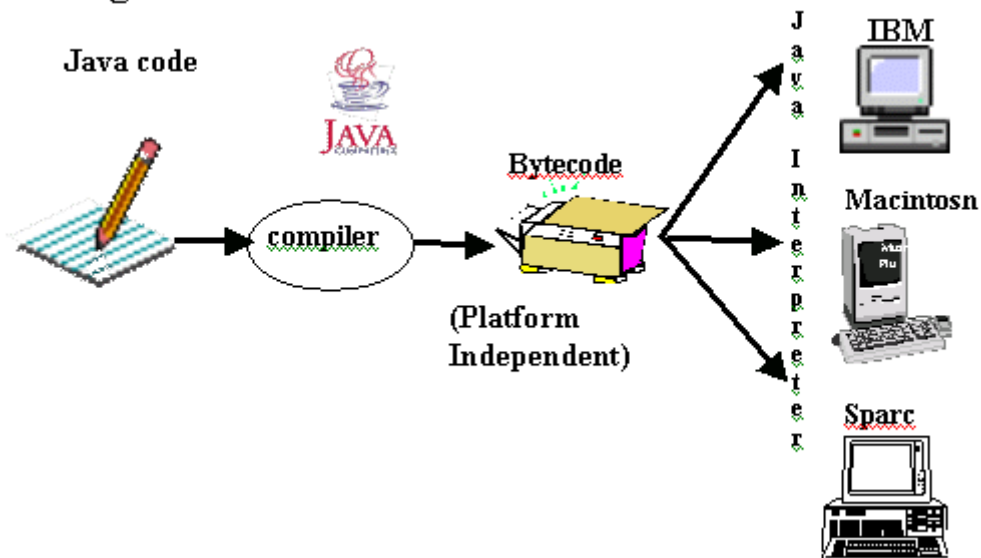
```

2.4. Biên dịch và thông dịch chương trình java

Quá trình dịch chương trình Java:

- Trình biên dịch chuyển mã nguồn thành tập các lệnh không phụ thuộc vào phần cứng cụ thể.
- Trình thông dịch trên mỗi máy chuyển tập lệnh này thành chương trình thực thi
- Máy ảo tạo ra một môi trường để thực thi các lệnh bằng cách:
 - Nạp các file .class
 - Quản lý bộ nhớ
 - Dọn “rác”

Programs



Hình 2.2 Chạy và biên dịch chương trình java

Chạy java chỉ cần cài máy ảo và khi chạy chỉ cần chạy file class, không cần phải dịch lại nên nó độc lập với phần cứng máy tính: “viết một lần chạy ở mọi nơi”

2.5. Các đặc trưng của chương trình ứng dụng Java

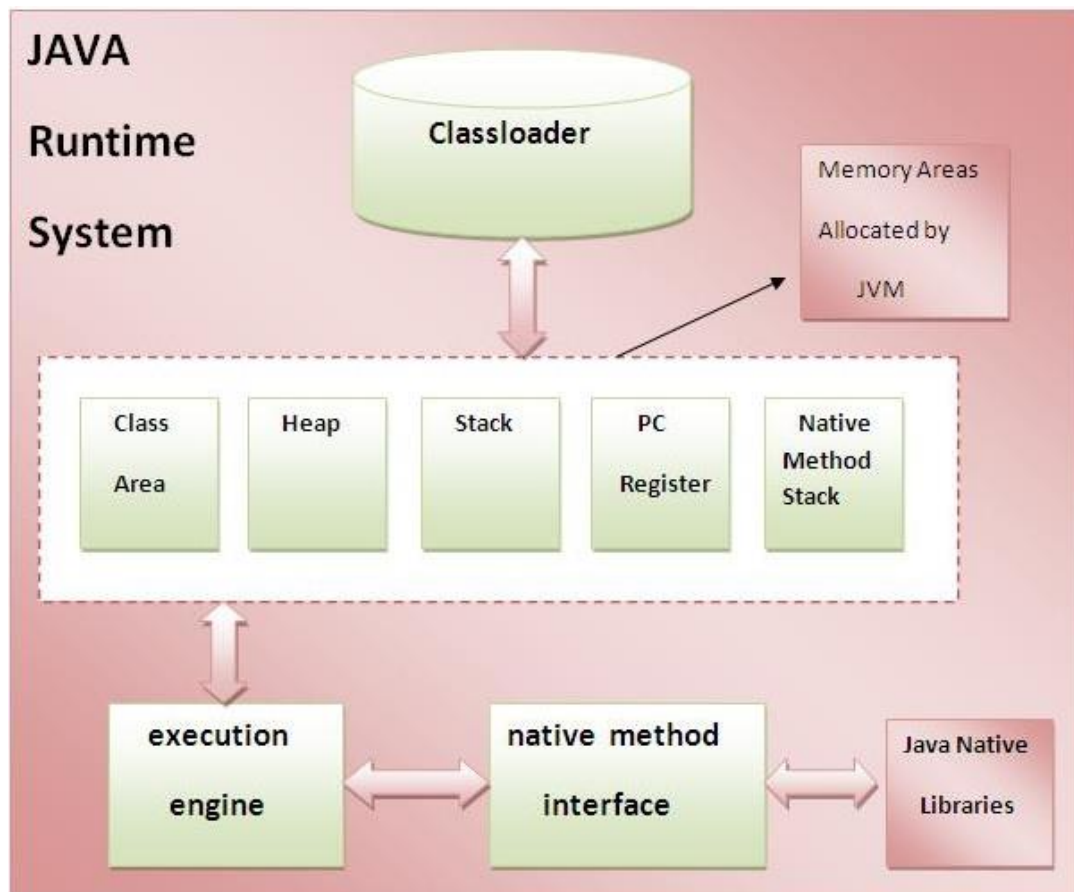
2.5.1. Máy ảo Java (JVM - Java Virtual Machine)

Tất cả các chương trình muốn thực thi được thì phải được biên dịch ra mã máy.

Mã máy của từng kiến trúc CPU của mỗi máy tính là khác nhau (tập lệnh mã máy của CPU Intel, CPU Solarix, CPU Macintosh ... là khác nhau), vì vậy trước đây một chương trình sau khi được biên dịch xong chỉ có thể chạy được trên một kiến trúc CPU cụ thể nào đó. Đối với CPU Intel chúng ta có thể chạy các hệ điều hành như Microsoft Windows, Unix, Linux, OS/2, ...

Chương trình thực thi được trên Windows được biên dịch dưới dạng file có đuôi .EXE còn trên Linux thì được biên dịch dưới dạng file có đuôi .ELF, vì vậy trước đây một chương trình chạy được trên Windows muốn chạy được trên hệ điều hành khác như Linux chẳng hạn thì phải chỉnh sửa và biên dịch lại.

Ngôn ngữ lập trình Java ra đời, nhờ vào máy ảo Java mà khó khăn nêu trên đã được khắc phục. Một chương trình viết bằng ngôn ngữ lập trình Java sẽ được biên dịch ra mã của máy ảo java (mã java bytecode). Sau đó máy ảo Java chịu trách nhiệm chuyển mã java bytecode thành mã máy tương ứng. Sun Microsystem chịu trách nhiệm phát triển các máy ảo Java chạy trên các hệ điều hành trên các kiến trúc CPU khác nhau.



Hình 2.3 Kiến trúc máy ảo Java

Trong đó:

- **Classloader:** Là một hệ thống con của JVM được sử dụng để tải class file.
- **Class (method) Area:** Lưu trữ cấu trúc mỗi lớp, chẳng hạn như hằng, trường, dữ liệu phương thức, code của phương thức, ...
- **Heap:** Nó là khu vực dữ liệu runtime mà trong đó đối tượng được cấp phát.
- **Stack:** Stack trong Java lưu giữ các Frame. Nó giữ các biến cục bộ và các kết quả cục bộ, và thực hiện một phần nhiệm vụ trong phân triệu hồi và trả về phương thức. Mỗi Thread có một Stack riêng, được tạo tại cùng thời điểm với Thread. Một Frame mới được tạo mỗi khi một phương thức được triệu hồi và bị hủy khi lời triệu hồi phương thức là kết thúc.
- **Program Counter Register:** Nó chứa địa chỉ của chỉ lệnh JVM hiện tại đang được thực thi.
- **Native Method Stack:** Bao gồm tất cả các phương thức tự nhiên được sử dụng trong ứng dụng.

- **Execution Engine:** Phần này bao gồm: Một bộ xử lý ảo Virtual Processor Một trình thông dịch Interpreter. Đọc Bytecode Stream sau đó thực thi các chỉ thị.
- **Just-In-Time (JIT) Compiler:** được sử dụng để cải thiện hiệu suất. JIT biên dịch các phần của Bytecode mà có cùng tính năng tại cùng một thời điểm, và vì thế giảm lượng thời gian cần thiết để biên dịch. Ở đây khái niệm Compiler là một bộ biên dịch tập chỉ thị của JVM thành tập chỉ thị của một CPU cụ thể.

2.5.2. Thông dịch

Java là một ngôn ngữ lập trình vừa biên dịch vừa thông dịch. Chương trình nguồn viết bằng ngôn ngữ lập trình Java có đuôi *.java đầu tiên được biên dịch thành tập tin có đuôi *.class và sau đó sẽ được trình thông dịch thông dịch thành mã máy.

2.5.3. Độc lập nền

Một chương trình viết bằng ngôn ngữ Java có thể chạy trên nhiều máy tính có hệ điều hành khác nhau (Windows, Unix, Linux, ...) với điều kiện ở đó có cài đặt máy ảo java (Java Virtual Machine).

2.5.4. Hướng đối tượng

Hướng đối tượng trong Java tương tự như C++ nhưng Java là một ngôn ngữ lập trình hướng đối tượng hoàn toàn. Tất cả mọi thứ đề cập đến trong Java đều liên quan đến các đối tượng được định nghĩa trước, thậm chí hàm chính của một chương trình viết bằng Java (đó là hàm main) cũng phải đặt bên trong một lớp. Hướng đối tượng trong Java không có tính đa kế thừa (multi inheritance) như trong C++ mà thay vào đó Java đưa ra khái niệm interface để hỗ trợ tính đa kế thừa.

2.5.5. Đa nhiệm - đa luồng (MultiTasking - Multithreading)

Java hỗ trợ lập trình đa nhiệm, đa luồng cho phép nhiều tiến trình, các tiến trình có thể chạy song song cùng một thời điểm và có thể tương tác với nhau.

2.5.6. Khả chuyển (portable)

Chương trình ứng dụng viết bằng ngôn ngữ Java chỉ cần chạy được trên máy ảo Java là có thể chạy được trên bất kỳ máy tính, hệ điều hành nào có máy ảo Java. “Viết một lần, chạy mọi nơi” (Write Once, Run Anywhere).

2.5.7. Hỗ trợ mạnh cho việc phát triển ứng dụng

Công nghệ Java phát triển mạnh mẽ nhờ vào “đại gia Sun Microsystem” cung cấp nhiều công cụ, thư viện lập trình phong phú hỗ trợ cho việc phát triển nhiều loại hình ứng dụng khác nhau cụ thể như:

- J2SE (Java 2 Standard Edition) hỗ trợ phát triển những ứng dụng đơn, ứng dụng client-server.

- J2EE (Java 2 Enterprise Edition) hỗ trợ phát triển các ứng dụng thương mại.

- J2ME (Java 2 Micro Edition) hỗ trợ phát triển các ứng dụng trên các thiết bị di động, không dây,...

2.6. Bộ công cụ phát triển JDK

JDK – Java Develop Kit – được dịch ra tiếng Việt là bộ công cụ phát triển ngôn ngữ lập trình bằng ứng dụng Java. JDK chính là một công nghệ và trong đó là tập hợp một loạt các công cụ nhằm hỗ trợ cho người dùng trong việc viết lên các chương trình ứng dụng dưới dạng ngôn ngữ lập trình. JDK dùng để viết ứng dụng mà trong đó gồm có JRE – dùng để chạy những ứng dụng mà JDK cần đến, để góp thêm phần làm ra những ứng dụng trong một hệ ngôn từ lập trình mà trong đó JDK có được sự tương hỗ từ những chương trình có sẵn trong JRE.

Từ khi ngôn từ lập trình Java sinh ra cũng đồng nghĩa tương quan với sự hình thành và tăng trưởng của bộ công cụ tương hỗ này ngay sau đó. Đây là một trong những bộ công cụ thông dụng và được sử dụng thoáng rộng và thông dụng bậc nhất đi cùng với sự tăng trưởng của ngôn từ lập trình JAVA. JDK cũng là một công nghệ tiên tiến được tăng trưởng bởi Sun Microsystem và được công bố cấp phép bởi GNU vào ngày 11/7/2006 – giấy phép về hội đồng GNU – một dạng giấy phép về ứng dụng tự do được cho phép người dùng hoàn toàn có thể san sẻ, sửa đổi và điều tra và nghiên cứu về những loại ứng dụng và được sử dụng rất thông dụng. JDK đã được cung ứng nguồn mở và trở thành một ứng dụng tự do. Nguồn mở không lấy phí – Openjdk được phát hành vào năm 2007 ngay sau đó.

Bộ dụng cụ trong JDK gồm:

- Trình tương hỗ thực thi những chương trình Java hay hiểu cách khác đây chính là môi trường tự nhiên để cho Java chạy và tăng trưởng. Nó chính là thiên nhiên và môi trường được tạo dựng nên để đem lại sự tăng trưởng cho những chương trình khác. Với tên gọi JRE (Java Runtime Environment).

- Trình biên dịch tương hỗ trong việc biên dịch ngôn từ lập trình từ ngôn từ nguồn (mã nguồn) sang ngôn từ đích. Thông thường những ngôn từ nguồn thường sẽ đơn thuần hơn những ngôn từ được biên dịch ra ngôn từ đích. (Java Compile).

- Trình tạo bản phân phối được cho phép toàn bộ những tệp thành một bản duy nhất việc làm này có tính năng rất lớn trong việc tiết kiệm chi phí dung tích trong quy

trình sử dụng những ứng dụng và việc tàng trữ tài liệu hay gửi đi một nơi khác trong thực tiễn trở nên nhỏ gọn và thuận tiện hơn. hay người ta gọi trình tạo bản phân phối này là Java Achieved – nén những file lại thành một bản duy nhất với những đuôi Jav một cách đồng nhất hóa.

- Trình sửa lỗi – Debug – chuyên đi nghiên cứu và phân tích và gỡ lỗi cho những chương trình Java.

- Trình tạo tài liệu – Java Doc

- Thư viện lập trình trong thực tiễn JDK = JRE + những công cụ tăng trưởng ứng dụng bên trong JRE như : Trình thông dịch, biên dịch, tàng trữ, trình sửa lỗi, ..

JRE - Java Runtime Environment – là môi trường để các ứng dụng thực thi trên đó trong khoảng thời gian lập trình. JRE có chứa các thư viện lớp Java, trình tải lớp và máy ảo Java.

Máy ảo Java, tiếng Anh là Java virtual machine (viết tắt: JVM) là một máy ảo cho phép chạy các chương trình Java cũng như các chương trình khác được viết bằng các ngôn ngữ khác mà được biên dịch sang mã byte Java. JVM được mô tả chi tiết bởi một đặc tả mô tả chính thức những gì cần cho một hiện thực JVM. Đặc tả đó để bảo đảm khả năng tương tác của các chương trình Java trên những hiện thực khác nhau để tác giả các chương trình đó có thể sử dụng Bộ công cụ Phát triển Java (JDK) mà không cần phải lo lắng về đặc tính của nền tảng phần cứng bên dưới.

2.7 Các môi trường tích hợp phát triển ứng dụng Java

Eclipse là một môi trường phát triển tích hợp dùng cho lập trình máy tính.[6], Nó chứa một không gian làm việc cơ sở và một hệ thống plug-in để mở rộng để tùy chỉnh môi trường. Eclipse được viết chủ yếu bằng Java và nó được dùng chủ yếu cho lập trình ứng dụng Java, nhưng nó cũng có thể dùng để lập trình ứng dụng bằng các ngôn ngữ khác thông qua plug-ins, bao gồm Ada, ABAP, C, C++, C#, Clojure, COBOL, D, Erlang, Fortran, Groovy, Haskell, HTML, JavaScript, Julia[7] Lasso, Lua, NATURAL, Perl, PHP, Prolog, Python, R, Ruby (Bao gồm Ruby on Rails framework), Rust, Scala, và Scheme. Nó cũng có thể dùng để phát triển các tài liệu bằng LaTeX (thông qua một plug-in TeXlipse) và các gói tin cho phần mềm Mathematica. Môi trường phát triển bao gồm Eclipse Java development tools (JDT) cho Java và Scala, Eclipse CDT cho C/C++, và Eclipse PDT for PHP, và những gói khác.

NetBeans là một môi trường phát triển tích hợp (IDE) cho Java. NetBeans cho phép các ứng dụng được phát triển từ một tập hợp các thành phần phần mềm được gọi

là modules. NetBeans chạy trên Windows, macOS, Linux và Solaris. Ngoài việc phát triển Java, nó còn có các phần mở rộng cho các ngôn ngữ khác như PHP, C, C++, HTML5, và JavaScript. Các ứng dụng dựa trên NetBeans, bao gồm NetBeans IDE, có thể được mở rộng bởi các nhà phát triển bên thứ ba.

jGRASP là phần mềm được cung cấp hỗ trợ cho môi trường phát triển tích hợp có thể tự động tạo các bản trình bày trực quan về phần mềm của nhà phát triển, cung cấp thông tin quan trọng liên quan đến cấu trúc, kích thước và các thuộc tính khác của phần mềm. Phần mềm có thể áp dụng cho môi trường Java, tương thích với mọi nền tảng Java. jGRASP có nhiều phiên bản khác nhau, dựa trên Hệ điều hành được cài đặt trên máy. Các biểu diễn trực quan do phần mềm tạo ra là Sơ đồ cấu trúc điều khiển, nhằm mục đích trực quan hóa mã nguồn; Sơ đồ lớp UML, nhằm mục đích trực quan hóa cấu trúc kiến trúc; và trình xem động, dành cho các chế độ xem thời gian chạy của các đối tượng như cây nhị phân; cũng như các chế độ xem thời gian chạy ban đầu. Điểm đặc biệt của phần mềm là Object Workbench, về cơ bản cho phép người dùng thêm một đối tượng vào workbench và phân tích các thay đổi trên đối tượng cho mọi phương thức mà người dùng gọi. Phần mềm cũng bao gồm một hướng dẫn có thể hướng dẫn người dùng mới làm quen với toàn bộ khái niệm về việc tạo tệp nguồn Java. Người dùng có thể học cách sử dụng đầy đủ từng tính năng của phần mềm và tìm hiểu tất cả các kiến thức cơ bản về tạo tệp nguồn Java.

2.8. Các dạng chương trình ứng dụng của Java

2.8.1. Chương trình ứng dụng độc lập

Ví dụ: gõ đoạn mã sau:

```
public class HelloWorldApp{  
    public static void main(String[] args){  
        System.out.println("HelloWorld");  
    }  
}
```

Lưu lại với tên HelloWorldApp.java

Để biên dịch mã nguồn, ta sử dụng trình biên dịch javac.

- Mở cửa sổ Command Prompt.
- Chuyển đến thư mục chứa tập tin nguồn vừa tạo ra.
- Thực hiện câu lệnh: **javac HelloWorldApp.java**

Trình dịch javac tạo ra file **HelloWordApp.class** chứa các mã “bytecodes”. Để chương trình thực thi được ta cần dùng trình thông dịch:

java HelloWorldApp

- Nếu chương trình đúng ta sẽ thấy dòng chữ HelloWorld trên màn hình Console.

- Nếu nhận được lỗi “Exception in thread "main

java.lang.NoClassDefFoundError:HelloWorldApp” có nghĩa là Java không thể tìm được tập tin mã bytecode tên HelloWorldApp.class.

Cấu trúc của chương trình **java HelloWorldApp**

```
public static void main(String args[ ])
```

Đây là phương thức chính, từ đây chương trình bắt đầu việc thực thi của mình. Tất cả các ứng dụng java đều sử dụng một phương thức main này.

- Từ khoá public là một chỉ định truy xuất. Nó cho biết thành viên của lớp có thể được truy xuất từ bất cứ đâu trong chương trình.

- Từ khoá static cho phép main được gọi tới mà không cần tạo ra một thể hiện (instance) của lớp. Nó không phụ thuộc vào các thể hiện của lớp được tạo ra.

- Từ khoá void thông báo cho máy tính biết rằng phương thức sẽ không trả lại bất cứ giá trị nào khi thực thi chương trình.

- String args[] là tham số dùng trong phương thức main. Khi không có một thông tin nào được chuyển vào main, phương thức được thực hiện với các dữ liệu rỗng – không có gì trong dấu ngoặc đơn.

- System.out.println(“Hello World”); Dòng lệnh này hiển thị chuỗi “Hello World” trên màn hình. Lệnh println() cho phép hiển thị chuỗi được truyền vào lên màn hình.

2.8.2. Chương trình ứng dụng nhúng

Applet là một chương trình Java có thể chạy trong các trình duyệt web có hỗ trợ Java. Tất cả các applet đều là các lớp con của lớp Applet. Để tạo applet, ta cần import hai gói sau:

```
import java.applet.*;
```

```
import java.awt.*;
```

- Cấu trúc của một Applet:

```
public class <Tên lớp applet> extends Applet{
```

```
... // Các thuộc tính
```

```
public void init(){...}
```

```
public void start(){...}
```

```

    public void stop(){...}
    public void destroy(){...}
    ... // Các phương thức khác
}

```

Các phương thức cơ bản của một applet:

- `init()`: Khởi tạo các tham số, nếu có, của applet.
- `start()`: Applet bắt đầu hoạt động
- `stop()`: Chấm dứt hoạt động của applet.
- `destroy()`: Thực hiện các thao tác dọn dẹp trước khi thoát khỏi applet.

Lưu ý:

- Không phải tất cả các applet đều phải cài đặt đầy đủ 4 phương thức cơ bản trên.

Applet còn có thể cài đặt một số phương thức tùy chọn (không bắt buộc) sau:

- `paint(Graphics)`: Phương thức vẽ các đối tượng giao diện bên trong applet. Các thao tác vẽ này được thực hiện bởi đối tượng đồ họa `Graphics` (là tham số đầu vào).
- `repaint()`: Dùng để vẽ lại các đối tượng trong applet. Phương thức này sẽ tự động gọi phương thức `update()`.
- `update(Graphics)`: Phương thức này được gọi sau khi thực hiện phương thức `paint` nhằm tăng hiệu quả vẽ. Phương này sẽ tự động gọi phương thức `paint()`.

Ví dụ: cài đặt một applet đơn giản vẽ ra chuỗi “hello word!”

```

import java.awt.*;
import java.applet.*;

public class SimpleApplet extends Applet{
    public void paint(Graphics g){
        g.drawString( " Hello world!",50,50);
    }
}

```

*** Sử dụng applet**

Applet không thể chạy như một ứng dụng Java độc lập (nó không có hàm `main`), mà nó chỉ chạy được khi được nhúng trong một trang HTML (đuôi `.htm`, `.html`) và chạy bằng một trình duyệt web thông thường.

Các bước xây dựng và sử dụng một applet bao gồm:

- Biên dịch mã nguồn thành lớp `.class`
- Nhúng mã `.class` của applet vào trang `html`.

Để nhúng một applet vào một trang html, ta dùng thẻ (tag) <Applet>:

Trong trang myHtml.htm có chứa nội dung như sau:

```
<HTML>
<HEAD>
<TITLE> A simple applet </TITLE>
</HEAD>
<BODY>
This is the output of applet:
<APPLET CODE = "SimpleApplet.class" WIDTH=200
HEIGHT=20>
</APPLET>
</BODY>
</HTML>
```

Mở trang myHtml trên các trình duyệt thông thường hoặc dùng lệnh appletviewer để chạy

2.9. Các phần tử cơ sở của Java

2.9.1. Các phần tử cơ sở

* Định danh (Tên gọi)

Trong Java định danh là một dãy các ký tự gồm các chữ cái, chữ số và một số các ký hiệu như: ký hiệu gạch dưới nối câu „_“, các ký hiệu tiền tệ \$, O, Ê, Â, và không được bắt đầu bằng chữ số.

Java phân biệt chữ thường và chữ hoa. Độ dài (số ký tự) của định danh trong Java về lý thuyết là không bị giới hạn.

Quy ước(lời khuyên):

- + Định danh được đặt tên cho các lớp, các đối tượng: chữ cái đầu của mỗi từ viết hoa.

- + Định danh cho các biến, phương thức, tên phương thức thường bằng động từ (viết in thường, và các danh từ phía sau thì chữ cái đầu viết chữ in hoa) và , đối tượng: chữ cái đầu của mỗi từ trong định danh đều viết hoa trừ từ đầu tiên(“náo”).

* Chú thích (Comment)

// Chú thích trên một dòng

/* Chú thích trên nhiều dòng

... */

/** Chú thích trong tư liệu (javadoc)

... */

*** Các từ khóa của java**

abstract	boolean	break	byte
case	catch	char	class
const	continue	default	do
double	else	extends	final
finally	float	for	goto
if	implements	import	instanceof
int	interface	long	native
new	package	private	protected
public	return	short	static
super	switch	synchronized	this
throw	throws	transient	try
void	volatile	while	

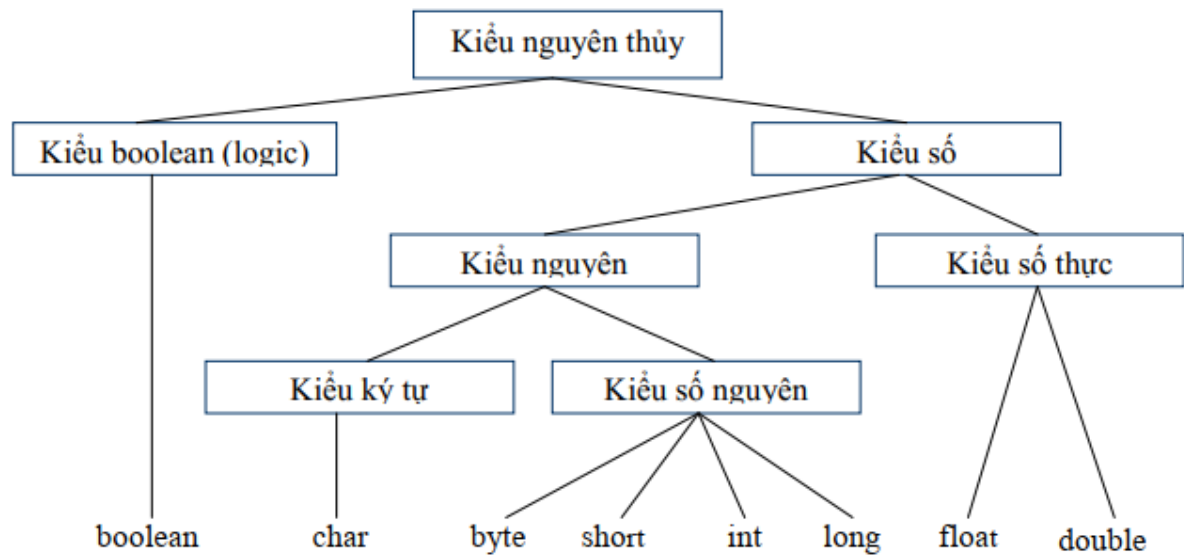
*** Các kí tự định dạng xuất dữ liệu (Escape Sequences)**

Escape Sequence	Mô tả
\n	Xuống dòng mới
\r	Chuyển con trỏ đến đầu dòng hiện hành
\t	Chuyển con trỏ đến vị trí dừng Tab kế tiếp (ký tự Tab)
\\	In dấu \
\'	In dấu nháy đơn (')
\"	In dấu nháy kép (")

2.9.2. Các kiểu dữ liệu trong Java

Java có 2 dạng dữ liệu: Dữ liệu nguyên thủy và dữ liệu tham chiếu đối tượng.

Các kiểu dữ liệu nguyên thủy là các kiểu dữ liệu có sẵn và định nghĩa trước của java:



Hình 2.4 Các kiểu dữ liệu

Mỗi kiểu nguyên thủy có một lớp bao bọc (Wrapper hay còn gọi lớp nguyên thủy) tương ứng để sử dụng các giá trị nguyên thủy như là các đối tượng. Ví dụ ứng với kiểu int có lớp Integer, ứng với char là Char vv.. các lớp này sẽ cung cấp một số phương thức hỗ trợ trên kiểu tương ứng,

Kiểu	Kích thước (bytes)	Giá trị min	Giá trị max	Giá trị mặc định
byte	1	-256	255	0
short	2	-32768	32767	0
int	4	-2^{31}	$2^{31} - 1$	0
long	8	-2^{63}	$2^{63} - 1$	0L
float	4			0.0f
double	8			0.0d

2.9.3. Khai báo biến

[phạm vi] <Kiểu dữ liệu> <tên biến>;

hoặc khai báo đồng thời khởi tạo giá trị ban đầu luôn:

[phạm vi]<Kiểu dữ liệu> <tên biến> = <giá trị>;

Lưu ý: chỉ khai báo biến thành phần của lớp mới có phạm vi truy cập.

<i>Kiểu dữ liệu</i>	<i>Giá trị mặc định</i>
boolean	false
char	'\u0000'
byte, short, int, long	0
float, double	+0.0F, +0.0D
Tham chiếu đối tượng	null

Khi khai báo: Nếu các biến không được khởi tạo giá trị tường minh thì:

- Các biến tĩnh luôn được khởi tạo với các giá trị mặc định.
- Các biến thành phần được khởi tạo mặc định mỗi khi đối tượng của lớp có thành phần đó được khởi tạo.
- Biến tham chiếu được gán mặc định là null nếu không tạo lập tường minh theo toán tử new và toán tử tạo lập (constructor).

Phạm vi hoạt động của biến

Một biến có phạm vi hoạt động là sau khi khai báo và trong toàn bộ khối lệnh mà nó được khai báo. Một khối lệnh bắt đầu bằng dấu "{" và kết thúc bằng dấu "}":

- ✓ Nếu biến được khai báo trong một cấu trúc lệnh điều khiển, biến đó có phạm vi hoạt động trong khối lệnh tương ứng.
- ✓ Nếu biến được khai báo trong một phương thức (Không nằm trong khối lệnh nào), biến đó có phạm vi hoạt động trong phương thức tương ứng: có thể được sử dụng trong tất cả các khối lệnh của phương thức.
- ✓ Nếu biến được khai báo trong một lớp (Không nằm trong trong một phương thức nào), biến đó có phạm vi hoạt động trong toàn bộ lớp tương ứng: có thể được sử dụng trong tất cả các phương thức của lớp.

2.9.4. Biểu thức trong Java

Số ưu tiên	Tên gọi	Các phép toán	Quy tắc kết hợp thực hiện
1	Phép toán 1 ngôi hậu tố (postfix)	[] . (tham_so) exp++ exp--	Từ trái qua phải
2	Phép toán 1 ngôi tiền tố (prefix)	++exp --exp +exp - exp ~ !	Từ phải qua trái
3	Tạo lập đối tượng, ép kiểu	new() (type)	Từ phải qua trái
4	Loại phép nhân	* / %	Từ trái qua phải

5	Loại phép cộng	+ -	Từ trái qua phải
6	Chuyển dịch	<< >> >>>	Từ trái qua phải
7	Phép toán quan hệ	< <= > >= instanceof	Từ trái qua phải
8	Phép so sánh đẳng thức	== !=	Từ trái qua phải
9	Phép AND	&	Từ trái qua phải
10	Phép XOR	^	Từ trái qua phải
11	Phép OR		Từ trái qua phải
12	Phép AND	&&	Từ trái qua phải
13	Phép hoặc (OR) logic		Từ trái qua phải
14	Phép toán điều kiện	?:	Từ trái qua phải
15	Các phép gán	= += -= *= /= %= <<= >>= >>>= &= ^= =	Từ phải qua trái

2.9.5. Các phép toán trong Java

- ✓ Phép chia nguyên (2 toán hạng đều là kiểu nguyên) đòi hỏi số chia phải khác 0.
- ✓ Phép chia số thực (ít nhất một toán hạng kiểu số thực) cho phép chia cho 0 và kết quả phép chia cho 0.0 là INF (số lớn vô cùng) hoặc -INF (số âm vô cùng), hai hằng đặc biệt trong Java.
- ✓ Trong Java, phép chia lấy số dư % thực hiện được cả đối với số thực. (float m = 11.5 % 2.5; // Cho m là 1.5)
- ✓ Khi sử dụng các phép toán đơn nguyên đối với các đối số kiểu byte, short, hoặc char thì trước tiên toán hạng phải được tính rồi chuyển về kiểu int và kết quả là kiểu int.

Ví dụ:

`short h = 30; // OK: 30 kiểu int chuyển về short (mặc định đối với hằng nguyên)`

`h = h + 4; // Lỗi vì h + 4 cho kết quả kiểu int không thể gán cho h kiểu short phải`

`ép kiểu: h = (short) (h+4);`

- ✓ Các hằng số thực trong Java được xem là giá trị (mặc định) kiểu *double*.

`float t = 3.14; // Lỗi vì không tương thích kiểu`

Vì thế, hoặc phải thông báo tường minh là số thực kiểu *float* (3.14F):

`float t = 3.14F; hoặc float t = (float)3.14;`

- ✓ Các phép chuyển dịch <<, >>, >>> thực hiện dịch dạng biểu diễn nhị phân của toán hạng thứ nhất sang trái, sang phải số lần bằng giá trị số nguyên của toán hạng thứ 2.
- ✓ Biểu diễn nhị phân của các số nguyên: Java sử dụng *phần bù 2 để lưu trữ các giá trị nguyên*.

Cho trước giá trị nguyên dương, ví dụ 41. Biểu diễn nhị phân của -41 được tính như sau:

	Biểu diễn nhị phân	Giá trị thập phân
Cho trước giá trị	00101001	41
Lấy phần bù 1	11010110	
Cộng thêm 1	00000001	
Kết quả là phần bù	211010111	-41

Phép dịch trái: <<

$a \ll n$ Dịch tất cả các bit của a sang trái n lần, điền số 0 vào bên phải. ($a \ll n$ tương đương với $a * 2^n$).

Phép dịch phải và điền bit dấu: >>

$a \gg n$ Dịch tất cả các bit của a sang phải n lần, điền bit dấu vào bên trái.

Phép dịch phải và điền bit 0: >>>

$a \ggg n$ Dịch tất cả các bit của a sang phải n lần, điền 0 vào bên trái. $A \ggg n$ tương đương với $a * 1/2^n$

- ✓ Giá trị của đối số bên phải (ở trên là n) luôn là số nguyên (dương) do vậy đối số bên trái (ở trên là a) nếu là *byte* hoặc *short* thì phải đổi sang kiểu *int*. Kết quả của các phép chuyển dịch vì vậy sẽ luôn là *int* (hoặc là kiểu *long* nếu đối số thứ nhất là *long*).
- ✓ Các phép gán số học mở rộng tương đương ngữ nghĩa với lệnh sau:

$\langle \text{var} \rangle = (\langle \text{type} \rangle) (\langle \text{var} \rangle \langle \text{op} \rangle (\langle \text{exp} \rangle));$

Trong đó $\langle \text{type} \rangle$ là các kiểu số và $\langle \text{op} \rangle$ là phép toán số học $+$, $-$, $*$, $/$, $\%$.

- ✓ So sánh đẳng thức trên các giá trị kiểu nguyên thủy: $==$, $!=$

Cho trước hai toán hạng a , b có kiểu dữ liệu nguyên thủy.

$a == b$ (a , b có giá trị kiểu nguyên thủy bằng nhau thì cho kết quả *true*, ngược lại cho *false*).

$a != b$ (a , b có các giá trị kiểu nguyên thủy không bằng nhau thì cho kết quả *true*, ngược lại cho *false*).

- ✓ So sánh đẳng thức trên các tham chiếu đối tượng: ==, !=

Cho trước r và s là hai biến tham chiếu.

`r == s` Cho giá trị true nếu r, s cùng tham chiếu tới cùng một trị (đối tượng), ngược lại sẽ cho giá trị false.

`r != s` Cho giá trị true nếu r, s không cùng tham chiếu tới cùng một trị (đối tượng), ngược lại sẽ cho giá trị false.

Chuyển đổi kiểu

- ép kiểu

Qui tắc ép kiểu có dạng: (**<type>**) **<exp>**

Chuyển kết quả tính toán của biểu thức **<exp>** sang kiểu được ép là **<type>**.

Ví dụ: `float f = (float) 100.15D; // Chuyển số 100.15 dạng kiểu double sang float`

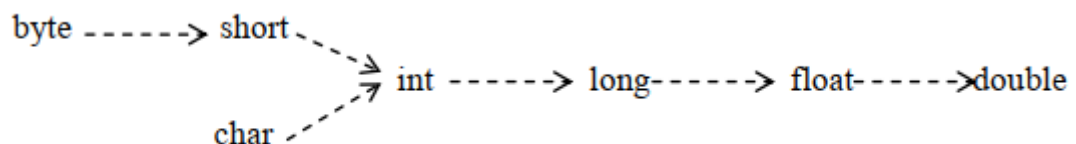
Lưu ý:

Không cho phép chuyển đổi giữa các kiểu nguyên thủy với kiểu tham chiếu, ví dụ kiểu *double* không thể ép sang các kiểu lớp như *HocSinh* được.

Kiểu giá trị *boolean* (logic) không thể chuyển sang các kiểu dữ liệu số và ngược lại.

- Mở rộng và thu hẹp kiểu

Giá trị của kiểu hẹp hơn (chiếm số byte ít hơn) có thể được chuyển sang những kiểu rộng hơn (chiếm số byte nhiều hơn) mà không tổn thất thông tin. Cách chuyển kiểu đó được gọi là mở rộng kiểu.



Ví dụ: `char c = "A";`

`int k = c; // mở rộng kiểu char sang kiểu int (mặc định)`

Chuyển đổi kiểu theo chiều ngược lại, từ kiểu rộng về kiểu hẹp hơn được gọi là *thu hẹp kiểu*. Lưu ý là *thu hẹp kiểu* có thể dẫn tới mất thông tin.

2.10. Các câu lệnh nhập xuất dữ liệu cơ bản qua các thiết bị vào ra chuẩn

2.10.1. Nhập dữ liệu từ bàn phím

Java là một ngôn ngữ lập trình thuần hướng đối tượng, nên cách nhập dữ liệu từ bàn phím cũng phải sử dụng theo hướng đối tượng. Tức là ta phải khai báo một đối tượng, sau đó gọi hàm nhập, rồi gán giá trị cho biến.

Scanner là một lớp trong thư viện Java.util giúp bạn nhập và lưu data từ bàn phím. Ngoài scanner, còn có nhiều lớp khác nữa, nhưng đây là lớp được sử dụng nhiều nhất.

Các phương thức thường dùng trong lớp **Scanner**:

Tên phương thức	Tác dụng
nextBoolean	Nhập vào kiểu Boolean (true – false) từ bàn phím
nextByte	Nhập vào kiểu dữ liệu Byte
nextShort	Nhập vào kiểu Short (số nguyên từ -32768 đến 32767)
nextInt	Nhập vào kiểu số nguyên từ bàn phím
nextFloat	Nhập vào kiểu số thực
nextDouble	Nhập vào kiểu Double
nextLine	Nhập vào kiểu String
nextLong	Nhập vào số nguyên lớn

Cú pháp

```
Scanner ip = new Scanner(System.in);
```

```
ip.nextInt(); // Gọi hàm nhập số nguyên. Các hàm khác cú pháp tương tự
```

2.10.2. Xuất dữ liệu ra màn hình

Cú pháp

```
System.out.println("Noi dung can in" + biểu thức);
```

```
System.out.print("Noi dung can in" + biểu thức);
```

```
System.out.printf("Noi dung can in" + biểu thức);
```

- ✓ Với *print*: Xuất kết quả ra màn hình nhưng con trỏ chuột không xuống dòng.
- ✓ Với *println*: Xuất kết quả ra màn hình đồng thời con trỏ chuột nhảy xuống dòng tiếp theo.
- ✓ Với *printf*: Xuất ra màn hình kết quả đồng thời có thể định dạng được kết quả đó nhờ vào các đối số thích hợp.

```
System.out.printf(local, format, arguments1, arguments2,..., argumentsn);
```

Trong đó:

- *local*: Nếu khác null sẽ được tự động định dạng theo khu vực.
- *format*: Quy định chuẩn định dạng đầu ra cho các đối số
- Các *argument*: Đối số cần định dạng.

Các bộ định dạng có sẵn trong printf:

%c: Ký tự
 %d: Số thập phân (số nguyên) (cơ số 10)
 %e: Dấu phẩy động theo cấp số nhân
 %f: Dấu phẩy động
 %i: Số nguyên (cơ sở 10)
 %o: Số bát phân (cơ sở 8)
 %s: Chuỗi
 %u: Số thập phân (số nguyên) không dấu
 %x: Số trong hệ thập lục phân (cơ sở 16)
 %t: Định dạng ngày / giờ
 %%: Dấu phần trăm
 \%: Dấu phần trăm

2.11. Các câu lệnh điều khiển

2.11.1. Câu lệnh rẽ nhánh

Câu lệnh if

```
if ( <Biểu thức điều kiện> )
    <lệnh 1>
[else <lệnh 2> ]
```

Nếu biểu thức điều kiện đúng thì thực hiện lệnh 1 nếu không thì thực hiện lệnh 2 (nếu có *else*)

Ví dụ:

```
public class Test {
    public static void main(String[] args) {
        int number = 13;
        if (number % 2 == 0) {
            System.out.println("Số " + number + " là số chẵn.");
        } else {
            System.out.println("Số " + number + " là số lẻ.");
        }
    }
}
```

Kết quả:

Số 13 là số lẻ.

Câu lệnh switch

```
switch (<Biểu thức nguyên>) {  
    case nhan_1: <Câu lệnh 1>  
    case nhan_2: <Câu lệnh 2>  
    ...  
    case nhan_n: <Câu lệnh n>  
    [default: <Câu lệnh >]  
}
```

`switch` sẽ kiểm tra giá trị của *Biểu thức nguyên* và so sánh với từng giá trị *nhan1*, *nhan2*, ..., lần lượt từ trên xuống dưới, mỗi giá trị cần so sánh gọi là một *case*. Khi một trường hợp đúng (true), khối lệnh ở trong *case* đó sẽ được thực thi.

Nếu tất cả các trường hợp đều sai (tức là các *nhan* ở *case* không bằng với *Biểu thức nguyên*), thì câu lệnh ở trong *default* sẽ được thực thi.

Ví dụ:

```
public class SwitchExample {  
    public static void main(String[] args) {  
        int number = 20;  
        switch (number) {  
            case 10:  
                System.out.println("10");  
                break;  
            case 20:  
                System.out.println("20");  
                break;  
            case 30:  
                System.out.println("30");  
                break;  
            default:  
                System.out.println("Not in 10, 20 or 30");  
        }  
    }  
}
```

Kết quả:

20

Lưu ý:

- Không *default* là không bắt buộc có ở cấu trúc *switch case* trong Java
- Trong một *switch* có thể có nhiều *case*

- Nếu không gặp lệnh *break* trong khối lệnh này, thì chương trình sẽ thực hiện tiếp các *case* bên dưới cho tới khi nó gặp lệnh *break* thì nó sẽ thoát ra khỏi *switch*, hoặc cho đến khi thực hiện hết câu lệnh.

Ví dụ: mệnh đề *Switch-case* khi không sử dụng '*break*'

```
public class SwitchExample2 {
    public static void main(String[] args) {
        int number = 20;
        switch (number) {
            case 10:
                System.out.println("10");
            case 20:
                System.out.println("20");
            case 30:
                System.out.println("30");
            default:
                System.out.println("Not in 10, 20 or 30");
        }
    }
}
```

Kết quả:

20

30

Not in 10, 20 or 30

2.11.2. Các câu lệnh lặp

* Vòng lặp **for**

for (<Biểu thức bắt đầu>; <Điều kiện lặp>; <Biểu thức gia tăng>)
<Khối lệnh>

Trong đó:

- *Biểu thức bắt đầu*: được thực thi đầu tiên, và chỉ một lần. Bước này cho phép khai báo và khởi tạo bất kỳ biến điều khiển vòng lặp. Tất cả các biến được khai báo trong <*Biểu thức bắt đầu*> đều là cục bộ trong khối thân của chu trình *for*.
- *Điều kiện lặp*: Nếu nó là *true*, phần *Khối lệnh* được thực thi. Nếu nó là *false*, phần *Khối lệnh* không thực thi và luồng điều khiển nhảy tới lệnh tiếp theo sau vòng lặp *for*.
- *Biểu thức gia tăng*: Thay đổi (tăng hoặc giảm) giá trị của biến điều khiển sau mỗi lần lặp

Ví dụ:

```
public class Main {  
    public static void main(String args[]) {  
        for (int i = 0; i < 5; i++) {  
            System.out.println("Vị trí thứ: " + i);  
        }  
    }  
}
```

Kết quả:

```
Vị trí thứ 0  
Vị trí thứ 1  
Vị trí thứ 2  
Vị trí thứ 3  
Vị trí thứ 4
```

Lưu ý:

- Các thành phần của chu trình for là tùy chọn. Một trong <Biểu thức bắt đầu>, <Biểu thức gia tăng>, <Điều kiện kết thúc> có thể trống. Trường hợp <Điều kiện kết thúc> là trống thì điều kiện lặp của chu trình được xem là true.
- for (; ;) được sử dụng để xây dựng chu trình lặp vô điều kiện.

*** Vòng lặp while**

```
while (<điều kiện>)  
    <khối lệnh>
```

Trong đó:

- *điều kiện*: Biểu thức *Boolean*, nó trả về giá trị *True* hoặc *False*. Vòng lặp sẽ tiếp tục cho đến khi nào giá trị *True* được trả về
- *khối lệnh*: Các câu lệnh được thực hiện nếu *điều kiện* nhận giá trị *True*

Ví dụ:

```
public class Main {  
    public static void main(String args[]) {  
        int i=0;  
        while(i<5){  
            System.out.println("Vị trí thứ "+i);  
            i++;  
        }  
    }  
}
```

Vòng lặp *while* kiểm tra điều kiện nếu $i < 5$ thì in ra vị trí i . Mỗi lần lặp, chúng ta tăng i thêm 1 đơn vị, do đó, sau 5 lần lặp, i có giá trị là 5 và vòng lặp sẽ dừng lại.

Kết quả:

```
Vị trí thứ 0  
Vị trí thứ 1  
Vị trí thứ 2  
Vị trí thứ 3  
Vị trí thứ 4
```

* Vòng lặp do...while

Vòng lặp do ... while là tương tự như Vòng lặp while, ngoại trừ rằng phần thân của vòng lặp do...while được bảo đảm thực thi ít nhất một lần. Nói cách khác, vòng lặp do ... while thực hiện phần thân vòng lặp trước khi kiểm tra điều kiện.

```
do{  
    <khởi lệnh>  
}while (<điều kiện>)
```

Trong đó:

- *điều kiện*: Biểu thức *Boolean*, nó trả về giá trị *True* hoặc *False*. Vòng lặp sẽ tiếp tục cho đến khi nào giá trị *True* được trả về
- *khởi lệnh*: Các câu lệnh được thực hiện 1 lần sau đó kiểm tra điều kiện, nếu đúng thì thực hiện tiếp và ngược lại.

Ví dụ:

```
public class Main {  
    public static void main(String args[]) {  
        int i = 0;  
        do {  
            System.out.println("Vị trí thứ : " + i);  
            i++;  
        } while (i < 5);  
    }  
}
```

Kết quả:

```
Vị trí thứ 0  
Vị trí thứ 1  
Vị trí thứ 2  
Vị trí thứ 3  
Vị trí thứ 4
```

2.11.3. Các lệnh chuyển vị trong java

Câu lệnh break:

Câu lệnh *break* được sử dụng trong các khối được gắn nhãn, trong các chu trình lặp (*for*, *while*, *do-while*) và câu lệnh *switch* để chuyển điều khiển thực hiện chương trình ra khỏi khối trong cùng chứa nó.

- Trong gắn nhãn: phần còn lại của khối bị bỏ qua
- Trong chu trình lặp: khi gặp lệnh *break* thì phần còn lại của thân chu trình được bỏ qua và kết thúc chu trình đó,
- Trong lệnh *switch*: phần còn lại của lệnh *switch* bị bỏ qua và tiếp tục thực hiện lệnh đứng sau lệnh *switch* đó.

Ví dụ:

```
public class BreakExample {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 10; i++) {  
            if (i == 5) {  
                break;  
            }  
            System.out.println(i);  
        }  
    }  
}
```

Kết quả:

```
1  
2  
3  
4
```

Ví dụ sử dụng *break* với vòng lặp bên trong vòng lặp *for* khác:

```
public class BreakExample2 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (i == 2 && j == 2) {  
                    break;  
                }  
                System.out.println(i + " " + j);  
            }  
        }  
    }  
}
```

```

        }
    }
}

```

Kết quả:

```

1 1
1 2
1 3
2 1
3 1
3 2
3 3

```

Câu lệnh continue

Câu lệnh continue được sử dụng trong các chu trình lặp for, while, do-while để dừng sự thực hiện của lần lặp hiện thời và bắt đầu lặp lại lần tiếp theo nếu điều kiện lặp còn thoả mãn (*true*).

Ví dụ sử dụng Continue trong java với vòng lặp for:

```

public class ContinueExample {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            if (i == 5) {
                continue;
            }
            // Khi i == 5 thì không in i = 5 ra màn hình
            System.out.println(i);
        }
    }
}

```

Kết quả:

```

1
2
3
4
6
7

```

8
9
10

Ví dụ sử dụng Continue với vòng lặp bên trong vòng lặp for khác:

```
public class ContinueExample2 {  
    public static void main(String[] args) {  
        for (int i = 1; i <= 3; i++) {  
            for (int j = 1; j <= 3; j++) {  
                if (i == 2 && j == 2) {  
                    continue;  
                }  
                // Không in trường hợp i=2 và j=2 ra màn hình  
                System.out.println(i + " " + j);  
            }  
        }  
    }  
}
```

Kết quả:

1 1
1 2
1 3
2 1
2 3
3 1
3 2
3 3

Câu lệnh return

Câu lệnh return được sử dụng để kết thúc thực hiện của hàm hiện thời và chuyển điều khiển chương trình về lại sau lời gọi hàm đó.

Dạng lệnh return	Hàm khai báo void	Hàm có kiểu trả lại khác void
return	Tùy chọn	Không cho phép
return <Biểu thức>	Không cho phép	Bắt buộc

2.12. Ngoại lệ và xử lý ngoại lệ

Mọi đoạn chương trình đều tiềm ẩn khả năng sinh lỗi. Lỗi chủ quan: do lập trình sai, lỗi khách quan: do dữ liệu, do trạng thái của hệ thống.

Ngoại lệ: các trường hợp hoạt động không bình thường của chương trình ví dụ: đọc 1 file không tồn tại, lỗi chia cho 0 với số nguyên, truy cập vượt quá phạm vi xác định của mảng vv... Xử lý ngoại lệ như thế nào? Làm thế nào để có thể tiếp tục thực hiện?

Cách xử lý lỗi truyền thống: Cài đặt mã xử lý tại nơi phát sinh ra lỗi, làm cho chương trình trở nên khó hiểu, không phải lúc nào cũng đầy đủ thông tin để xử lý, không nhất thiết phải xử lý, truyền trạng thái lên mức trên, thông qua tham số, giá trị trả lại hoặc biến tổng thể (flag). Dễ nhầm, khó kiểm soát được hết các trường hợp, lỗi số học, lỗi bộ nhớ,... Lập trình viên thường quên không xử lý lỗi do bản chất con người, thiếu kinh nghiệm, cố tình bỏ qua.

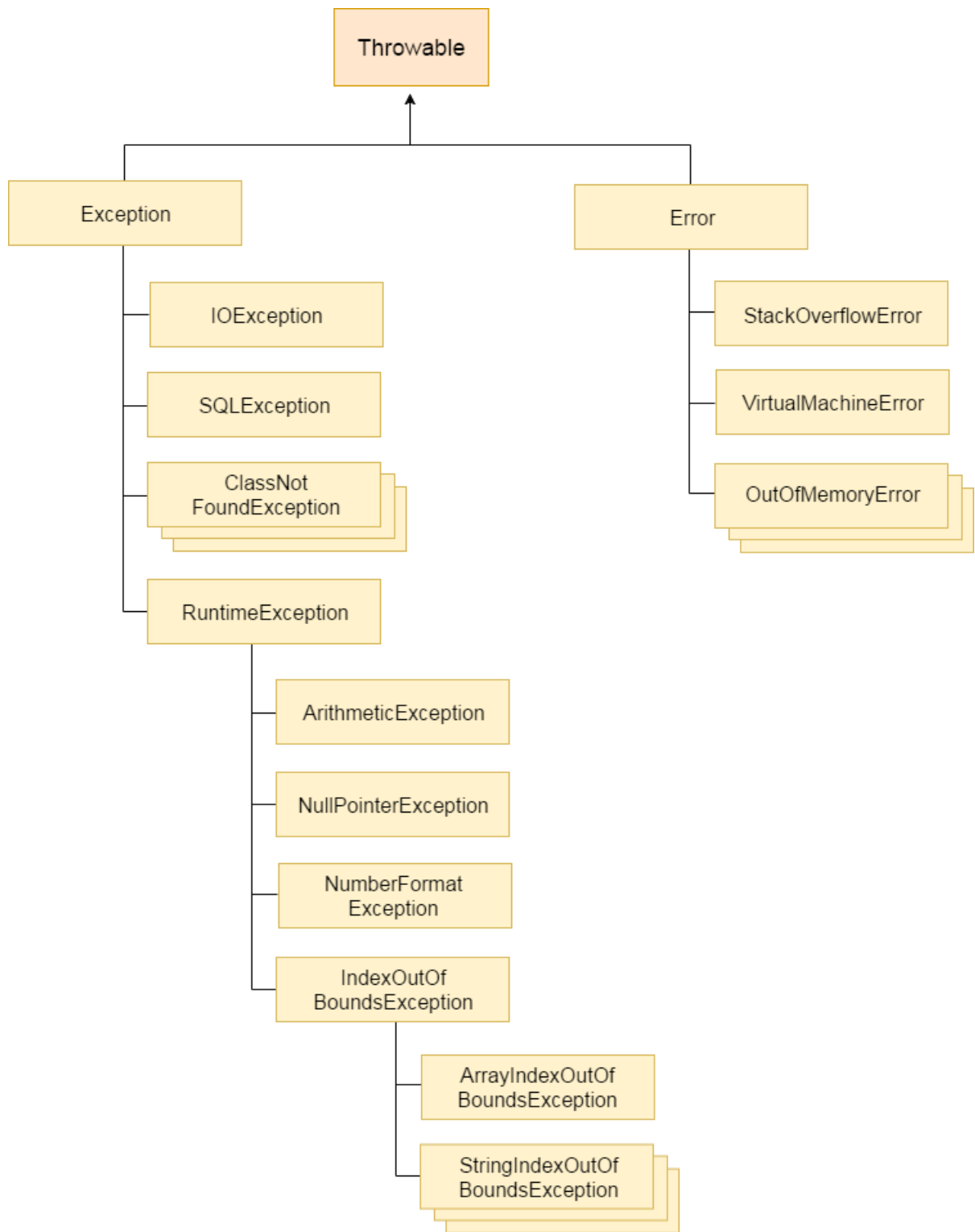
Ví dụ, việc chia cho 0 sẽ tạo một lỗi trong chương trình hoặc khi xét thao tác nhập xuất (I/O) trong một tập tin. Nếu việc chuyển đổi kiểu dữ liệu không thực hiện đúng, một ngoại lệ sẽ xảy ra và chương trình bị hủy mà không đóng lại tập tin. Lúc đó tập tin dễ bị hư hại và các nguồn tài nguyên được cấp phát cho tập tin không được thu hồi lại cho hệ thống.

Ngôn ngữ Java cung cơ chế tự động dùng cho bắt và xử lý ngoại lệ. Có hai cách để xử lý ngoại lệ: Bắt, xử lý ngoại lệ và *ném* ngoại lệ (chuyển hướng hay ủy quyền xử lý ngoại lệ đến một vị trí khác của chương trình).

Ưu điểm của bắt ngoại lệ

- Dễ sử dụng
- Dễ dàng chuyển điều khiển đến nơi có khả năng xử lý ngoại lệ
- Có thể ném nhiều loại ngoại lệ
- Tách xử lý ngoại lệ khỏi thuật toán
- Tách mã xử lý
- Sử dụng cú pháp khác
- Không bỏ sót ngoại lệ (ném tự động)
- Làm chương trình dễ đọc hơn, an toàn hơn

2.12.1. Cấu trúc phân cấp của các lớp xử lý ngoại lệ



Hình 2.5 Ngoại lệ

Ngoại lệ	Lớp cha của thứ tự phân cấp ngoại lệ
RuntimeException	Lớp cơ sở cho nhiều ngoại lệ java.lang
ArithmeticException	Trạng thái lỗi về số, ví dụ như „chia cho 0“
IllegalAccessException	Lớp không thể truy cập
IllegalArgumentException	Phương thức nhận một đối số không hợp lệ

ArrayIndexOutOfBoundsException	Kích thước của mảng lớn hơn 0 hay lớn hơn kích thước thật sự của mảng
NullPointerException	Khi muốn truy cập đối tượng null
SecurityException	Việc thiết lập cơ chế bảo mật không được hoạt động
ClassNotFoundException	Không thể nạp lớp yêu cầu
NumberFormatException	Việc chuyển đổi không thành công từ chuỗi sang số thực
AWTException	Ngoại lệ về AWT
IOException	Lớp cha của các ngoại lệ I/O
FileNotFoundException	Không thể định vị tập tin
EOFException	Kết thúc một tập tin
NoSuchMethodException	Phương thức yêu cầu không tồn tại
InterruptedException	Khi một luồng bị ngắt

Bảng. 2.1 Ngoại lệ

2.12.2. Câu lệnh *try*, *catch* và *finally*

try { // Khối *try*

<Các câu lệnh> }

catch (<Kiểu ngoại lệ 1> <Tham biến 1>) { // Khối *catch*

<Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ 1> }

...

catch (<Kiểu ngoại lệ n> <Tham biến n>) { // Khối *catch*

<Các câu lệnh xử lý khi xuất hiện kiểu ngoại lệ n>

}

finally { < Các câu lệnh phải thực hiện đến cùng> // Khối *finally*}

Trong đó:

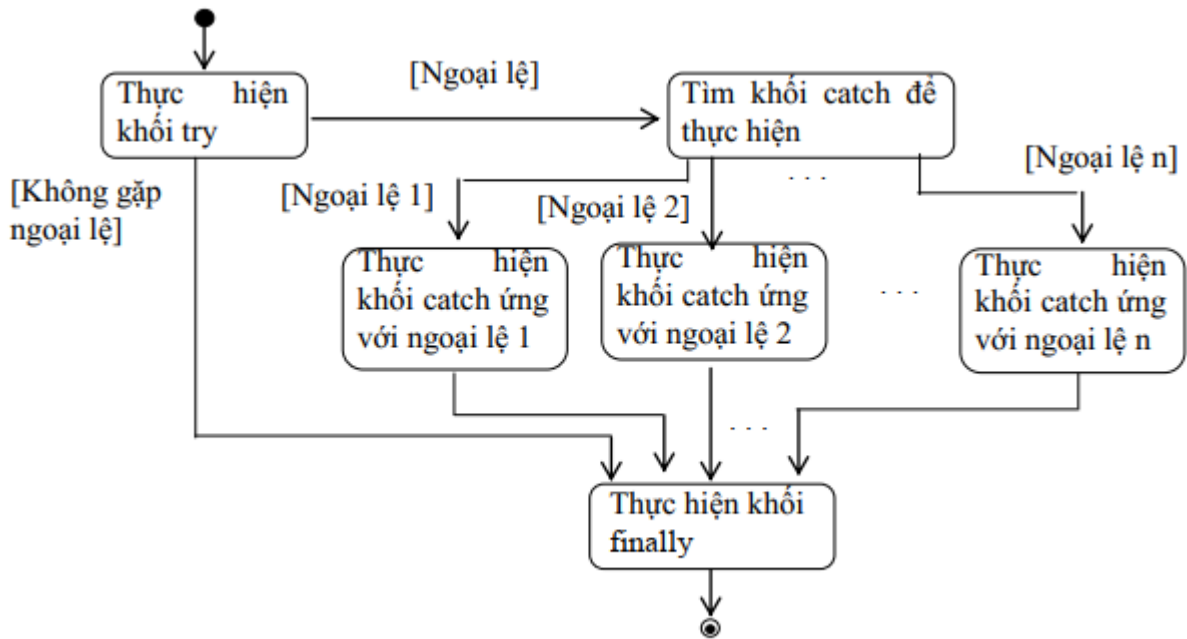
- <Các câu lệnh> là các câu lệnh có khả năng gây ra ngoại lệ

- <Kiểu ngoại lệ> là các kiểu ngoại lệ tương ứng.

<tham biến i> là đối tượng mang thông tin về ngoại lệ

Các ngoại lệ sẽ được cho qua trong quá trình thực hiện khối *try* và sẽ bị tóm lại để xử lý ở các khối *catch* tương ứng nếu có. Khối *finally* phải thực hiện đến cùng, bất luận có gặp phải ngoại lệ hay không.

Hoạt động của các khối trên được minh hoạ như sau:



Hình 2.6. Try catch

Khối try

Một chương trình cũng có thể chứa các khối *try* lồng nhau. Khi sử dụng các “*try*” lồng nhau, khối *try* bên trong được thi hành đầu tiên. Bất kỳ ngoại lệ nào bị chặn trong khối *try* sẽ bị bắt giữ trong các khối *catch* theo sau. Nếu khối *catch* thích hợp không được tìm thấy thì các khối *catch* của các khối *try* bên ngoài sẽ được xem xét. Nếu không, Java Runtime Environment xử lý các ngoại lệ.

Khối catch

Khối này chỉ được sử dụng để xử lý ngoại lệ nếu có. Khi một khối *catch* được thực hiện thì các khối *catch* còn lại sẽ bị bỏ qua.

Có thể bắt nhiều loại ngoại lệ khác nhau bằng cách sử dụng nhiều khối lệnh *catch* đặt kế tiếp

Khối lệnh *catch* sau không thể bắt ngoại lệ là lớp dẫn xuất của ngoại lệ được bắt trong khối lệnh *catch* trước.

Khối finally

Khi một ngoại lệ xuất hiện, phương thức đang được thực thi có thể bị dừng mà không được thi hành toàn vẹn. Nếu điều này xảy ra, thì các đoạn mã sẽ không bao giờ được gọi. Khối *finally* thực hiện tất cả các việc thu dọn khi một ngoại lệ xảy ra. Khối *finally* bảo đảm lúc nào cũng được thực thi, bất chấp có ngoại lệ xảy ra hay không. Khối *finally* là tùy ý, không bắt buộc.

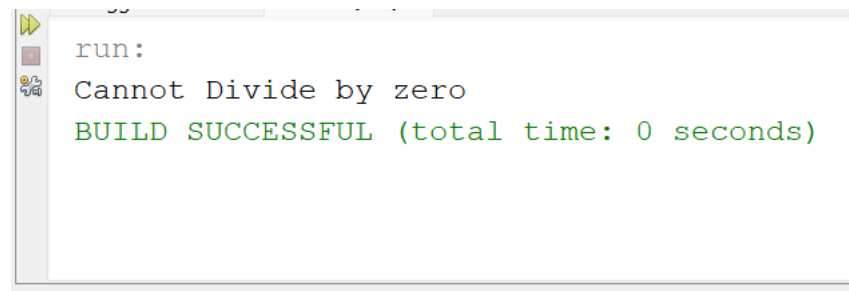
Khởi *finally* có thể dùng để:

- Đóng tập tin.
- Đóng lại bộ kết quả (sử dụng trong chương trình cơ sở dữ liệu).
- Đóng lại các kết nối được tạo trong cơ sở dữ liệu.

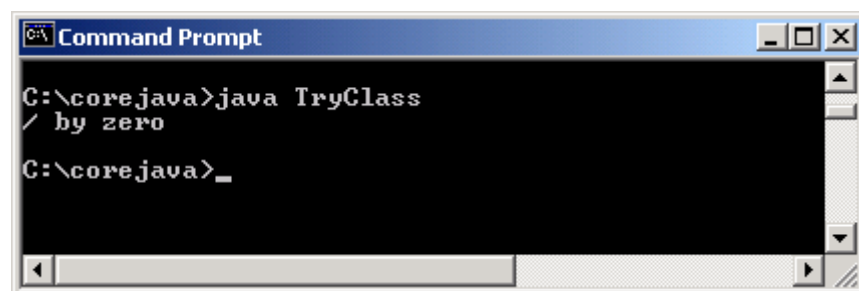
Ví dụ minh họa cách sử dụng các khối *try* và *catch*.

```
class TryClass{  
    public static void main(String args[])  
    {  
        int demo=0;  
        try { System.out.println(20/demo); }  
        catch(ArithmeticException a) {  
            System.out.println("Cannot Divide by zero"); }  
    }  
}
```

Kết quả:



a được sử dụng như một đối tượng của *ArithmeticException* để in các chi tiết về các toán tử ngoại lệ mà hệ thống cung cấp. Nếu bạn thay thế lệnh *System.out.println* của khối *catch* bằng lệnh *System.out.println(a.getMessage())* thì kết quả của chương trình sẽ như sau:



2.12.3. Ném ngoại lệ bằng lệnh 'throw'

Để tạm thời bỏ qua ngoại lệ chúng ta có thể sử dụng câu lệnh *throw*. Đoạn lệnh sau chỉ ra cách sử dụng của lệnh *throw*:

```
try{ if (flag<0)
    {throw new MyException();} // user-defined
}
```

Ví dụ:

```
if (0==denominator) {
    throw new Exception();
} else res = nominator / denominator;
```

Giả sử phương thức $x()$ gọi phương thức $y()$. Phương thức $y()$ chặn một ngoại lệ không được xử lý. Phương thức gọi $x()$ nên khai báo việc chặn cùng một ngoại lệ với phương thức được gọi $y()$. Ta nên khai báo khối *try catch* trong phương thức $x()$ để đảm bảo rằng ngoại lệ không được truyền cho các phương thức mà gọi phương thức này.

Phương thức được định nghĩa lại tại lớp dẫn xuất có thể không ném ngoại lệ. Nếu ném ngoại lệ, chỉ có thể ném ngoại lệ giống tại phương thức của lớp cơ sở hoặc ngoại lệ là lớp dẫn xuất của ngoại lệ được ném tại phương thức của lớp cơ sở.

Ví dụ:

```
class A {public void methodA() throws RuntimeException { }
}
class B extends A {
    public void methodA() throws ArithmeticException { }
}
class C extends A {
    public void methodA() throws Exception { }
}
class D extends A {
    public void methodA() { }
}
A a = new B();
try { a.methodA();}
catch (RuntimeException e) { ...}
```

Ném lại ngoại lệ: Sau khi bắt ngoại lệ, nếu thấy cần thiết chúng ta có thể ném lại chính ngoại lệ vừa bắt được để cho chương trình mức trên tiếp tục xử lý.

```
try {...}
catch (Exception e) {
    System.out.println(e.getMessage());
    throw e;}
}
```

2.12.4. Mệnh đề *throws*

Khi thiết kế các hàm thành phần, chúng ta có thể sử dụng mệnh đề *throws* để tạm thời cho qua ngoại lệ mà thực hiện một số công việc cần thiết khác. Chúng ta ném ngoại lệ khỏi phương thức khi:

- Không nhất thiết phải xử lý ngoại lệ trong phương thức
- Không đủ thông tin để xử lý
- Không đủ thẩm quyền

Định nghĩa hàm với mệnh đề *throws* có dạng:

<Thuộc tính của hàm> <tên hàm>(<Danh sách các tham biến>)

throws <Danh sách các kiểu ngoại lệ> { /* ... */ }

Trong đó <Danh sách các kiểu ngoại lệ> là các lớp xử lý ngoại lệ được kế thừa từ lớp *Exception* và được phân tách bởi dấu ‘,’.

Ném ngoại lệ từ *main()*

- Nếu không có phương thức nào bắt ngoại lệ, ngoại lệ sẽ được truyền lên phương thức *main()* và được cần được xử lý tại đây.
- Nếu vẫn không muốn xử lý ngoại lệ, chúng ta có thể để ngoại lệ truyền lên mức điều khiển của máy ảo bằng cách khai báo *main()* ném ngoại lệ
- chương trình sẽ bị dừng và hệ thống sẽ in thông tin về ngoại lệ trên Console (*printStackTrace()*)

2.12.5. Tự định nghĩa ngoại lệ

Chúng ta có thể tạo lớp ngoại lệ để phục vụ các mục đích riêng

- Lớp ngoại lệ mới phải kế thừa từ lớp *Exception* hoặc lớp dẫn xuất của lớp này.
- Có thể cung cấp hai constructor
- constructor nhận một tham số *String* và truyền tham số này cho phương thức khởi tạo của lớp cơ sở.

Ví dụ:

```
class SimpleException extends Exception { }  
class MyException extends Exception {  
    public MyException() {}  
    public MyException(String msg) {  
        super(msg);  
    }  
}
```

Mình họa ngoại lệ được định nghĩa bởi người dùng *ArraySizeException*:

```

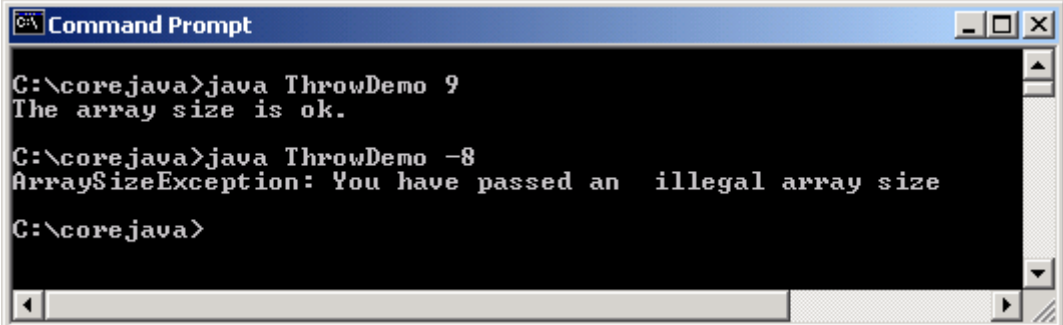
class ArraySizeException extends NegativeArraySizeException
{ ArraySizeException() // constructor
    { super("You have passed an illegal array size");
    }
}

class ThrowDemo
{ int size, array[];
  ThrowDemo(int s)
  { size=s;
  try { checkSize(); }
  catch(ArraySizeException e){ System.out.println(e);}
  }
  void checkSize() throws ArraySizeException
  { if (size < 0)
    throw new ArraySizeException();
  else
    System.out.println("The array size is ok.");
  array = new int[3];
  for (int i=0; i<3; i++) array[i] = i+1;
  }
  public static void main(String arg[])
  { new ThrowDemo(Integer.parseInt(arg[0])); }
  }

```

Lớp được định nghĩa bởi người dùng *ArraySizeException* là lớp con của lớp *NegativeArraySizeException*. Khi một đối tượng được tạo từ lớp này, thông báo về ngoại lệ được in ra. Phương thức *checkSize()* được gọi để chặn ngoại lệ *ArraySizeException* mà được chỉ ra bởi mệnh đề *throws*. Kích thước của mảng được kiểm tra trong cấu trúc *if*. Nếu kích thước là số âm thì đối tượng của lớp *ArraySizeException* được tạo. Phương thức *call()* được bao quanh trong khối *trycatch*, là nơi mà giá trị của đối tượng được in ra. Phương thức *call()* cần được bao trong khối *try*, để cho khối *catch* tương ứng có thể in ra giá trị.

Kết xuất của chương trình:



```

C:\core.java>java ThrowDemo 9
The array size is ok.

C:\core.java>java ThrowDemo -8
ArraySizeException: You have passed an illegal array size

C:\core.java>

```

2.12.6. Lăn vết ngoại lệ *StackTrace*

Có thể sử dụng phương thức *printStackTrace()* để lần vết vị trí phát sinh ngoại lệ debug chương trình

```
public class Test4 {  
    void methodA() throws Exception {  
        methodB();  
        throw new Exception();  
    }  
    void methodB() throws Exception {  
        methodC();  
        throw new Exception();}  
    void methodC() throws Exception {  
        throw new Exception();}  
    public static void main(String[] args) {  
        Test4 t = new Test4();  
        try { t.methodA(); }  
        catch(Exception e) {  
            e.printStackTrace();  
        }  
    }  
}
```

Bài tập luyện tập

2.1. Câu nào sau đây là sai?

- A. `/** chú thích */`
- B. `/* chú thích */`
- C. `/* chú thích`
- D. `// chú thích`

2.2. Phương thức `nextLine()` thuộc lớp nào ?

- A. `String`
- B. `Scanner`
- C. `Integer`
- D. `System`

2.3. Kiểu dữ liệu nào trong Java chứa giá trị bao gồm cả chữ và số?

- A. `int`
- B. `byte`
- C. `char`
- D. `String`

2.4. Đây là khai báo biến hợp lệ?

- A. `theOne`
- B. `the One`
- C. `1the_One`
- D. `$the One`

2.5. Biểu thức nào có giá trị khác các biểu thức còn lại trong các biểu thức sau? Cho `x=true` thuộc kiểu `boolean`.

- A. `true`
- B. `x==true;`
- C. `1==1`
- D. `!x`

2.6. Có bao nhiêu kiểu dữ liệu cơ sở trong Java?

- A. 7
- B. 8
- C. 9
- D. 5

2.7. Có bao nhiêu kiểu số nguyên trong Java?

- A. 1
- B. 2
- C. 3
- D. 4

2.8. Khối lệnh sau có kết quả bao nhiêu?

```
String greeting = "Hello";  
int k = greeting.length();  
System.out.print(k);
```

- A. 4
- B. 5
- C. 6

D. 7

2.9. Kết quả in ra của đoạn mã bên dưới là gì? Chọn một câu trả lời

```
public class Test {  
    public static int switchIt(int x)  
    {  
        int j = 1;  
        switch (x) {  
            case 1:  
                j++;  
            case 2:  
                j++;  
            case 3:  
                j++;  
            case 4:  
                j++;  
            default:  
                j++;  
        }  
        return j + x;  
    }  
    public static void main(String[] args) {  
        System.out.println("value = " + switchIt(4));  
    }  
}
```

A. value = 3

B. value = 4

C. value = 5

D. value = 7

2.10. Đây là khai báo đúng về lớp Cat?

A. Class Cat{}

B. **class public** Cat(){}

C. **class** Cat{}

D. **public** Cat **class**{}

2.11. Giá trị của x sau khi thực hiện lệnh trên sẽ cho kết quả gì?

Khi thực hiện lệnh:

A a = **new** A();

a.calTotal(a);

```
class A {  
    int x = 10;  
    public void calTotal(A a) {  
        a.x = 12;  
        System.out.println(a.x);  
    }  
}
```

A. 10

- B. 12
- C. 22
- D. 44

Bài tập mẫu (có hướng dẫn cách giải): Kiểm tra giá trị của biến c

```
/*
Tên file IfDemo.java.
*/
class IfDemo {
    public static void main(String args[]) {
        int a, b, c;
        a = 2;
        b = 3;
        if (a < b) {
            System.out.println("a is less than b");
        }
        // this won't display anything
        if (a == b) {
            System.out.println("you won't see this");
        }
        System.out.println();
        c = a - b; // c contains -1
        System.out.println("c contains -1");
        if (c >= 0) {
            System.out.println("c is non-negative");
        }
        if (c < 0) {
            System.out.println("c is negative");
        }
        System.out.println();
        c = b - a; // c now contains 1
        System.out.println("c contains 1");
        if (c >= 0) {
            System.out.println("c is non-negative");
        }
        if (c < 0) {
            System.out.println("c is negative");
        }
    }
}
```

Bài tập thực hành:

Bài 1: Viết chương trình nhập vào 3 số nguyên từ bàn phím, chương trình sẽ in ra số nguyên có giá trị lớn nhất

Bài 2: Viết chương trình nhập và 1 số nguyên từ bàn phím, chương trình sẽ in ra thông tin số vừa nhập là số ‘âm’, ‘dương’ hoặc là số ‘không’

Bài 3: Viết chương trình cho phép nhập vào 3 số thực tương ứng với độ dài của một tam giác và kiểm tra 3 số này có phải là 3 cạnh của một tam giác hay không(nếu nó thỏa mãn điều kiện tổng của hai cạnh bất kỳ đều lớn hơn cạnh thứ 3) .

Bài 4: Với n nhập từ bàn phím hay viết các chương trình tính tổng các dãy sau:

1. Tính tổng dãy

$$S=1+2-3+.....(-1)^{n+1}.n$$

2. Tính tổng dãy:

$$S=1!+2!+3!+...+n!$$

3. Tính tổng các số lẻ <n, nếu n chẵn: tính tổng các số chẵn <n.

$$4. S = \frac{1}{2} + \frac{2}{3} + + \frac{n-1}{n}$$

Bài 5 : Viết chương trình cho phép nhập vào 3 số thực tương ứng với độ dài của một tam giác và kiểm tra 3 số này có phải là 3 cạnh của một tam giác vuông hay không (nó là một tam giác, và bình phương của cạnh huyền bằng tổng bình phương của hai cạnh còn lại) .

Bài 6 : Dùng vòng lặp *for* viết chương trình tìm n số Fibonacci đầu tiên trong java. Số nguyên dương n được nhập từ bàn phím.

Tương tự với yêu cầu như trên nhưng dùng các cấu trúc *while*, do *while*

Quy luật của dãy số Fibonacci: số tiếp theo bằng tổng của 2 số trước, 2 số đầu tiên của dãy số là 0, 1. Ví dụ: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...